Atividade Integradora

Nome: Antonio Leonardo | Vinicius Falcão
Prof: Paulo Nunes | Laboratório POO

Parte I — Questões teóricas

Bloco A — Fundamentos de POO

1. Explique a diferença entre classe e objeto. Apresente um exemplo de cada um em um sistema acadêmico.

R: A classe é o molde que define as características e os comportamentos de algo, enquanto um objeto é uma instância concreta criada a partir dessa classe.

Exemplo em sistema acadêmico: Classe aluno. A classe aluno define as informações e ações que um aluno pode ter em um sistema acadêmico.

- **Atributos: Nome, matrícula e curso.**
- **Métodos: Consultar notas e verificar frequência.**
-

Objeto Aluno1
Um objeto criado a partir da classe aluno poderia representar um estudante específico

- **Nome: Leonardo**
- **Matrícula: 2025001**
- **Curso: Engenharia de Software**

2. O que são atributos e métodos? Diferencie estado e comportamento de um objeto.

R: Na programação atributos e métodos são elementos que definem as características e ações de um objeto.
Atributos: São as informações ou características de um objeto
Métodos: São as ações ou operações que um objeto pode executar.

Estado: Corresponde aos valores atuais dos atributos de um objeto.
Comportamento: Corresponde às ações que o objeto pode realizar por meio de seus métodos.

3. Qual é a função de new? Diferencie criar um objeto e declarar uma referência.

R: Em Java a palavra-chave new é utilizada para criar um objeto na memória por meio da instanciação de uma classe. Ela reserva espaço pro objeto e retorna uma referência que permite acessar. A declaração de uma referência cria uma variável capaz de armazenar referência a um objeto, mas não cria o objeto em si.

4. Explique encapsulamento. Por que evitar atributos públicos sem necessidade?

R: Encapsulamento consiste em proteger os dados de um objeto controlando o acesso a seus atributos por meio de métodos e modificadores de acesso, como private e public. Devemos evitar atributos públicos sem necessidade porque eles permitem que outras partes do programa modifiquem os dado diretamente

5. Qual é a finalidade do construtor? Explique o uso de this.

R: É um membro especial executado durante a criação do objeto. Ele possui o mesmo nome da classe e não declara tipo de retorno, seu papel típico é colocar um objeto em um estado inicial válido.This representa o objeto, isto é, a instância cujo método ou construtor está sendo executado. É especialmente útil quando o parâmetro possui o mesmo nome do atributo.

Bloco B — Relacionamentos e herança
6. Diferencie associação, agregação e composição; exemplifique.

R: Associação é um relacionamento em que um objeto conhece ou utiliza. Os objetos podem continuar existindo independentemente.
Ex: Livro e autor. Se o objeto livro deixar de existir o autor pode continuar existindo.

Agregação representa uma relação todo-parte mais fraca: um objeto agrupa outros objetos, mas as partes possuem ciclo de vida independente.
Ex: Uma biblioteca pode agregar que existe independentemente daquela biblioteca

Composição representa uma relação todo=parte mais forte: Didaticamente usamos a ideia de que um objeto todo controla fortemente a criação e o ciclo de vida das partes.
Ex: Um pedido pode criar e controlar seus ItemPedido.

7. O que é ArrayList e quando preferi-lo a um vetor de tamanho fixo?
R: É uma coleção dinâmica da biblioteca java. Pode usar o ArrayList quando a quantidade de dados for desconhecida, variar com frequência ou precisar de manipulação dinâmica.

8. Diferencie superclasse e subclasse. Explique extends e super.
R: Superclasse é a classe geral que fornece atributos e métodos para as outras classes. Subclasse que herda atributos e métodos de uma superclasse.
Extends: Palavra-chave usada para indicar que uma classe herda de outra.
Super: Palavra-chave usada para acessar membros da superclasse ou chamar seu construtor.

9. Diferencie sobrecarga e sobrescrita de métodos; dê exemplos.
R: Sobrecarga ocorre quando existem métodos ou construtores com o mesmo nome, mas listas de parâmetros diferentes.
EX: Somar(2, 3) soma dois números; soma(2, 3, 4) soma três números.

Sobrescrita redefine em uma subclasse um comportamento herdado. EX: O método emitirSom() da classe Cachorro substitui o comportamento herdado de animal.

10. O que é polimorfismo? Como uma referência de superclasse aponta para objetos de subclasses?
R: Tem como objetivo compreender como classes podem compartilhar características e como objetos de subclasses podem ser tratados por um tipo mais geral. Em um sistema de registro de funcionários, as variáveis f1 e f2 são do tipo Funcionario. Porém, cada referência aponta para uma subclasse diferente. Quando calcularSalario() é chamado, Java seleciona em tempo de execução a implementação correspondente ao objeto real.

Bloco C — Interfaces e projeto
11. O que é interface? Diferencie implements e extends nos exemplos estudados.
R: Uma interface descreve um contrato que diferentes classes podem implementar. Extends estabelece herança entre classes; implements declara que uma classe cumpre o contrato de uma interface.

12. Compare herança e composição. Quando a composição é preferível?
R: Herança permite criar uma classe mais específica a partir de uma classe mais geral enquanto composição representa uma classe que possui objetos de outra classe. A composição é preferível quando queremos combinar funcionalidades sem criar uma hierarquia de herança.

13. O que é baixo acoplamento? Como a injeção por construtor ajuda?
R: Acontece quando as classes dependem pouco uma das outras, facilitando alterações e manutenção do código. Injeção por

construtor ajuda recebendo suas dependências pelo construtor, em vez de criá-las diretamente dentro dela. Isso permite trocar implementações e facilita os testes.

14. Explique SRP, OCP e DIP com exemplos simples.
R: SRP: O princípio da responsabilidade única orienta que uma classe mantenha uma responsabilidade coerente e uma razão principal para mudar, não significa que uma classe deva ter apenas um método.
EX: Uma classe Pedido pode misturar responsabilidades entre calcularTotal, salvarBanco, enviarEmail, gerarPDF.

OCP: O princípio Aberto/Fechado busca permitir extensão de comportamento sem exigir alterações recorrentes em código estável. Uma interface pode funcionar como ponto de extensão.
EX: Uma nova classe chamada NotiicadorPush pode ser adicionada sem modificar a classe principal ServicoNotificacao, pois o serviço já trabalha com a abstração Notificador.

DIP: O Princípio da Inversão de Dependência orienta que módulos de alto nível não fiquem presos diretamente a detalhes concretos; ambos devem depender de abstrações apropriadas.
EX: ServicoNotificacao depende de Notificador, e não de NotificadorEmail

Bloco D — Padrões de projeto
15. O que é um padrão de projeto? Por que evitar aplicá-lo indiscriminadamente?
R: É uma solução reutilizável e testada para problemas comuns que ocorrem durante o desenvolvimento de software, funcionando como guias de arquitetura para tornar o código mais fácil de manter. Evitar indiscriminadamente para não gerar código difícil de entender e

manter para uma solução simples ou evitar engessar o sistema que tornam futuras mudanças mais custosas.

16. Explique Singleton e a finalidade do construtor privado e de getInstancia().
R: É um padrão de projeto criacional que garante que uma classe tenha apenas uma única instância em toda a aplicação e fornece um ponto de acesso global a ela. Construtor privado impede que qualquer código externo crie novas instâncias de classe usando o operador new, garantindo o controle absoluto de que o objeto pode ser gerado dentro da própria classe. O método getInstancia() é o ponto de acesso global controlado. Ele verifica se a única instância da classe já foi criada: se não foi, ela cria. Se já foi, ele retorna a instância existente.

17. Explique Factory Method e o papel dos criadores abstratos e concretos.
R: É um padrão de projeto criacional que fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo exato de objeto que será criado.
Criador abstrato: Define a interface ou classe base que declara o Factory Method. Geralmente contém a lógica de negócios geral da aplicação que precisa criar produtos.
Criador concreto: É a subclasse que herda do criador abstrato e sobrescrever o Factory Method para retornar uma instância de um produto específico. Cada criador concreto é responsável por fabricar um tipo específico de objeto.

18. Como Strategy organiza descontos comuns, estudantis e promocionais?
R: Organiza descontos separando eles em classes independentes, cada uma implementando uma interface comum definindo um contrato padrão enquanto a classe principal recebe a estratégia

desejada em tempo de execução e faz o cálculo dela sem precisar conhecer os detalhes de como o desconto é calculado.

19. Como Observer comunica um pedido confirmado a estoque, financeiro e auditoria?
R: Observer estabelece uma relação de um para muitos, onde um objeto central avisa automaticamente a vários outros interessados sempre que seu estado muda. O Assunto é o objeto Pedido, que mantém uma lista interna de todos os módulos que desejam ser avisados sobre suas mudanças de estado.
Os Observadores são o Estoque, Financeiro e Auditoria. Eles se "inscrevem" no Pedido previamente.

20. Diferencie Adapter e Facade; dê um caso de uso de cada um.
R: Adapter: Foca na compatibilidade de interfaces. Atua como um "tradutor" que converte a interface de uma classe existem em outra interface que o cliente espera encontrar.
Uso: Integrar um sistema legado de pagamento que utiliza uma biblioteca antiga com métodos próprios em um sistema moderno que exige a interface padronizada atual.

Facade: Foca na simplicidade de uso. Fornece uma interface unificada e simplificada para um subsistema complexo com várias classes, escondendo sua complexidade geral do cliente.
Uso: Simplificar o processo de finalização de compra de um e-commerce. Em vez de o cliente ter que interagir manualmente com o subsistema de estoque, gateway de pagamento, cálculo de frete e nota fiscal separadamente, ele chama um único método.

Etapa 8

1- Qual classe implementa Singleton e quais instruções controlam a instância?

R: A classe Configuracao. Ela é controlada por um atributo privado estático (private static final Configuracao INSTANCIA = new Configuracao()), um construtor privado (private Configuracao() {}) que impede novas instâncias com new, e um método estático público (getInstancia()) que retorna a única instância existente.

2- Onde aparecem encapsulamento e validação?

R: Nas classes Curso e Aluno, onde os atributos são marcados como private e a leitura é controlada por getters. As validações aparecem nos construtores e métodos de modificação (verificando se strings são nulas/vazias com .isBlank() e se preços são menores ou iguais a zero).

3- Qual é a relação entre Aluno e Curso?

R: É uma associação (especificamente uma agregação conceitual), onde a classe Aluno mantém uma coleção (ArrayList<Curso>) de cursos nos quais está matriculado.

4- Onde foi utilizada uma coleção e com qual objetivo?

R: O ArrayList foi utilizado na classe Aluno para armazenar múltiplos cursos e no ServicoMatricula para armazenar múltiplos observadores inscritos.

5- Qual é o papel da interface Desconto?

R: Estabelecer um contrato padrão (aplicar(double)). Ela permite o uso de polimorfismo, fazendo com que o serviço de matrícula processe qualquer política de desconto sem precisar conhecer suas implementações concretas (baixo acoplamento).

6- Como acrescentar uma política de 20% sem alterar o serviço?

R: Basta criar uma nova classe (ex: DescontoVintePorCento) que implemente a interface Desconto. Graças ao Princípio

Aberto/Fechado (OCP), o ServicoMatricula não precisará ser modificado.

7- Quem produz o evento e quem recebe as notificações?

R: O ServicoMatricula produz o evento ao concluir a matrícula, e as classes observadoras (AvisoEmail e Auditoria) recebem e tratam as notificações.

8- Por que o adaptador converte reais em centavos?

R: Porque o sistema legado aceita apenas valores inteiros em centavos, enquanto a arquitetura moderna da aplicação opera utilizando valores decimais em reais (double).

9- Qual método representa Factory Method?

R: O método abstrato criar() na classe CriadorComprovante, que é sobrescrito nas subclasses concretas (CriadorTexto e CriadorResumido) para instanciar e retornar o tipo correto de comprovante.

10- O que FachadaAcademica simplifica para Main?

R: Ela esconde toda a complexidade de orquestração interna (chamar o cálculo de desconto, registrar o pagamento no adaptador, matricular o aluno, notificar observadores e acionar o factory method de comprovantes), expondo para a classe Main apenas uma operação limpa e direta: matricularEEmitir().